UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

Deep Learning for NLP

Student name: *Kleanthis Vasileiadis*
sdi: 1115202200017

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *spring Semester 2025*

Contents

1 Abstract **2**

2 Data processing and analysis **2**
2.1 Pre-processing . . . 2
2.2 Analysis . . . 3
2.3 Data partitioning for train, test and validation . . . 4
2.4 Vectorization . . . 4

3 Algorithms and Experiments **5**
3.1 Experiments . . . 5
3.1.1 Experiment 1 - Baseline model . . . 5
3.1.2 Experiment 2 - Text pre-processing . . . 6
3.1.3 Experiment 3 - Number of Hidden Layers . . . 7
3.1.4 Experiment 4 – Number of Neurons per Hidden Layer . . . 9
3.1.5 Experiment 5 – Number of Epochs . . . 11
3.1.6 Experiment 6 – Learning Rate and Weight Decay . . . 14
3.1.7 Table of trials . . . 15
3.2 Hyper-parameter tuning . . . 16
3.2.1 Key Hyper-parameters . . . 16
3.2.2 Hyper-parameters Tuning Results . . . 17
3.2.3 Prevention of overfitting and underfitting . . . 18
3.3 Optimization techniques . . . 18
3.4 Evaluation . . . 18
3.4.1 ROC curve . . . 19
3.4.2 Learning Curve . . . 20
3.4.3 Classification report . . . 21
3.4.4 Confusion matrix . . . 22

4 Results and Overall Analysis **23**
4.1 Results Analysis . . . 23
4.1.1 Best trial . . . 24
4.2 Comparison with the first project . . . 24

5 Bibliography **25**

1. Abstract

This project is a continuation of Project 1 and aims to develop a sentiment classification model using machine learning and artificial intelligence techniques in Python, applied to a given English-language Twitter dataset. The dataset remains the same as in the previous project and consists of three main components: an ID that uniquely identifies each tweet, the text containing the tweet's content, and a sentiment label, where a value of 1 indicates a positive sentiment and 0 indicates a negative one. The goal of the classification process is to assign each tweet to one of these two sentiment categories.

In the previous project, classification was performed using the TF-IDF vectorization method in combination with a logistic regression machine learning model. In the current project, neural networks will be employed instead. Specifically, Word2Vec embeddings will be used as input to a feedforward neural network. The performance of this new approach will then be compared to the results obtained in Project 1.

2. Data processing and analysis

2.1. Pre-processing

Pre-processing is a crucial step in machine learning and AI applications, as it ensures that the input data is clean, consistent, and suitable for training a sentiment classification model. In this project, the same text preprocessing techniques as those used in the previous project were applied. Alternative methods were tested, and some of the original techniques were temporarily removed, however these changes consistently led to a decline in performance metrics.
The text preprocessing techniques employed are briefly described below:

- **Replacing emojis**: Converts emojis into the word "emoji".
- **Replacing emoticons**: Converts emoticons into their corresponding text representations.
- **Replacing HTML entities**: Converts HTML entities into readable text or their corresponding special character.
- **Replacing mentions**: Replaces mentions with the word "mention".
- **Replacing links**: Replaces URLs starting with "http","https" and "www" with the word "link".
- **Replacing money amounts**: Replaces money values with the word "money".
- **Replacing dates and months**: Converts standard date format(day/month/year) into the word "date" and month names into the word "month".
- **Replacing emails**: Replaces email addresses with the word "email".
- **Replacing phone numbers**: Replaces phone numbers of various formats(e.g., with country codes, dashes, or spaces) with the word "phone".
- **Replacing credit card numbers**: Replaces credit card numbers(13 to 16 digits, with optional spaces or dashes) with the word "creditcard".

- **Replacing IP addresses**: Replaces IP addresses(in the format xxx.xxx.xxx.xxx) with the word "ip".
- **Replacing time expressions**: Replaces time formats (e.g., "12:30 PM") with the word "time".
- **Replacing numbers**: Replaces the remaining numbers with the word "number".
- **Replacing names**: Replaces names with the word "name".
- **Replacing places**: Replaces named places (e.g.,countries, cities) with the word "place".
- **Replacing terrains**: Replacing geographic features (e.g., mountains, rivers) with "terrain".
- **Removing special characters**: Keeps only alphabetic characters and spaces.
- **Restricting repeated characters**: Restricting repeated consecutive characters to a maximum of two (e.g.,"looooooove"->"loove").
- **Normalizing text to lowercase**: Converts all text to lowercase for consistency.
- **Handling negations**: Detects and processes negations properly.
- **Tokenization**: Splits text into individual words.

The process of replacing emoticons, HTML entities, names, places, and terrains was carried out using custom lists and dictionaries, which were carefully constructed by analyzing the training dataset. All other replacements were performed using regular expressions (regex) with the "re" library. The handling of negation was done manually by identifying specific negation words extracted from the training dataset and appending a "not_" prefix to the subsequent word. Proper pre-processing not only standardizes the text but also helps reduce noise in the data, making overfitting less likely in the classification model.

2.2. Analysis

Before proceeding with model training, it is essential to analyze and visualize the dataset to gain insights into its structure and characteristics. This step involves exploring the raw text data, identifying patterns, and understanding the distribution of words and sentences. By performing this analysis both before and after preprocessing, we can evaluate the impact of text cleaning techniques and ensure that important information is preserved while noise is reduced.

Since the same dataset and text preprocessing techniques were utilized as in the previous project, an in-depth exploratory data analysis will not be repeated in this report. Nonetheless, it is important to highlight several key characteristics of the dataset:

- The dataset contains a total of 211,983 tweets. Of these, approximently 191K tweets are labeled.

- The labeled portion of the dataset is balanced, with an approximately equal number of positive and negative samples, which contributes to unbiased model training and evaluation.
- There are no missing values in any of the dataset's components, ensuring data completeness.

2.3. Data partitioning for train, test and validation

The dataset was provided pre-partitioned into training, validation, and test sets. The distribution follows a standard and well-balanced ratio: approximately 70 percent of the data (149K tweets) is used for training, 20 percent (42K tweets) for validation, and 10 percent (22K tweets) for testing.

This split ensures a well-balanced approach, allowing the model to learn effectively from a large portion of the data while keeping sufficient samples for hyperparameter tuning and evaluation. The validation set helps in preventing overfitting by guiding model adjustments, while the independent test set provides a final assessment of performance. These ratios are widely used in deep learning tasks, particularly for large NLP datasets, ensuring both reliable training and robust evaluation.

2.4. Vectorization

To enable the model to process textual data, the tweet content must first be transformed into a numerical format through a process known as vectorization. In this project, pre-trained GloVe Twitter embeddings with 200 dimensions were used to represent words in a dense vector space. Since GloVe embeddings are originally stored in a format not directly compatible with the Word2Vec interface, they were first converted into Word2Vec format to allow for easy integration with the embedding layer.

Word embeddings, such as those produced by Word2Vec and GloVe, are dense vector representations that capture semantic and syntactic relationships between words. Unlike traditional one-hot or TF-IDF representations, embeddings encode contextual similarity, meaning words that appear in similar contexts tend to have similar vector representations. This allows the model to generalize better and learn more abstract relationships.

In this implementation, each tweet was first tokenized into individual words. The corresponding 200-dimensional embedding vector was retrieved for each word from the pre-trained GloVe vocabulary. Words not present in the vocabulary were represented using a special out-of-vocabulary (OOV) token to ensure robustness when encountering unseen or rare words. Padding was also applied to standardize the length of all input sequences, enabling efficient batch processing by the neural network.

Before feeding the embeddings into the neural network, all vector values were normalized using a MinMaxScaler, which scaled the features to the fixed range of (0,1). This step helped ensure numerical stability during training and contributed to faster convergence and improved performance by maintaining consistent value ranges across the input vectors. The (0,1) range was chosen specifically to align well with activation functions such as the sigmoid, and to prevent the risk of exploding or vanishing gradients during backpropagation.

The final embedding matrix used in the model had a shape of `[65570, 200]`, meaning it included 65,570 unique tokens (including special tokens like padding and OOV), each mapped to a 200-dimensional vector. Out of these, 47,801 tokens were successfully matched to pre-trained embeddings, resulting in a vocabulary coverage of approximately 72.90%. The remaining 27.10% of tokens were either rare, misspelled, or domain-specific terms that did not exist in the original GloVe Twitter corpus.

3. Algorithms and Experiments

3.1. Experiments

Building an effective classification model requires iterative experimentation to refine its performance. This section details the approach taken to optimize the model, starting with an initial baseline run followed by systematic improvements through various techniques. The goal was to identify methods that enhance generalization, improve accuracy, and mitigate overfitting.

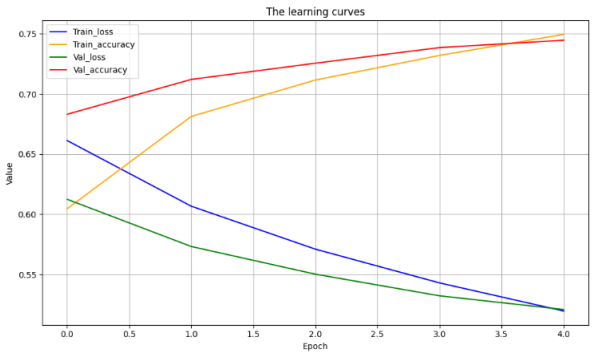
As in the previous project, a large number of experiments were conducted throughout the development process. Rather than presenting each one in detail, this report focuses on a selection of representative experiments that highlight the impact of various optimization techniques. These include both successful and unsuccessful attempts, as understanding what didn't work is just as important as what did. Each experiment is evaluated using a range of metrics, including learning curves, confusion matrices, ROC curves, and classification reports, to provide a comprehensive view of model performance.

3.1.1. Experiment 1 - Baseline model. The first experiment involved training a simple baseline model consisting of a single hidden layer with 200 neurons. No text preprocessing or hyperparameter tuning was applied at this stage, in order to establish a basic performance benchmark. The model was trained using the pre-trained GloVe Twitter embeddings as described earlier. As expected, due to the absence of any optimization or preprocessing, the model showed signs of underfitting and struggled to capture meaningful patterns from the data.

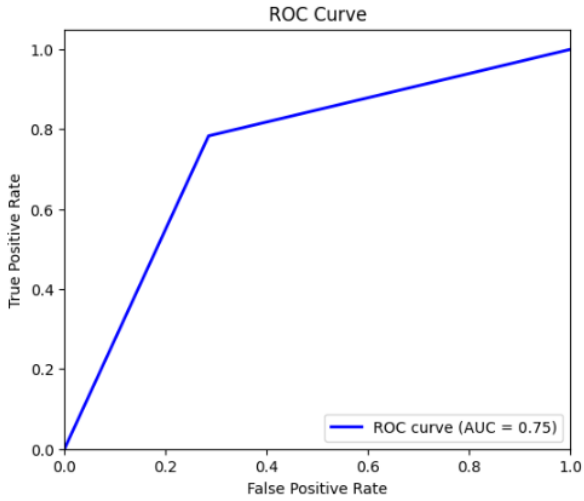
This initial experiment serves as a reference point, providing insight into the dataset's learnability and the model's raw performance before applying any optimizations. By establishing this baseline, we can later compare the impact of different improvements such as text preprocessing, hyperparameter tuning, feature selection, and optimization techniques. The baseline model achieved an accuracy of 74,95% on the training dataset and 74,46% on the validation dataset. While these results indicate that the model is already capturing some patterns in the data, there is still room for improvement.

	precision	recall	f1-score	support
0	0.81	0.72	0.76	24097
1	0.68	0.78	0.73	18299
accuracy			0.74	42396
macro avg	0.74	0.75	0.74	42396
weighted avg	0.75	0.74	0.75	42396

(a) Classification Report And Confusion Matrix



(b) Learning curve



(c) ROC curve

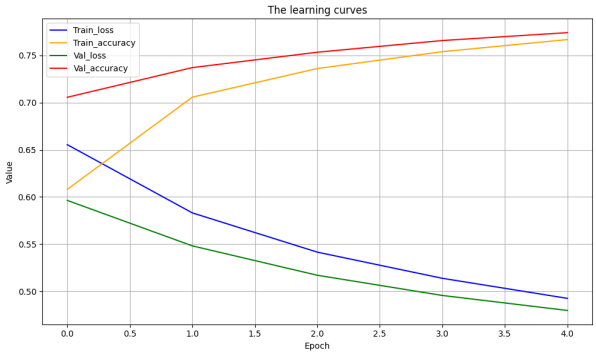
3.1.2. Experiment 2 - Text pre-processing. In Experiment 2, i incorporated text pre-processing techniques to enhance the model's performance by reducing noise and helping the model focus on the most relevant features of the data. This experiment builds upon the baseline model from Experiment 1, with the key difference being the inclusion of various text cleaning and transformation steps. These pre-processing steps, as outlined in Section 2, aim to standardize the text data, remove unnecessary information, and better prepare it for the deep learning model.

The model achieved noticeably improved performance compared to the baseline,

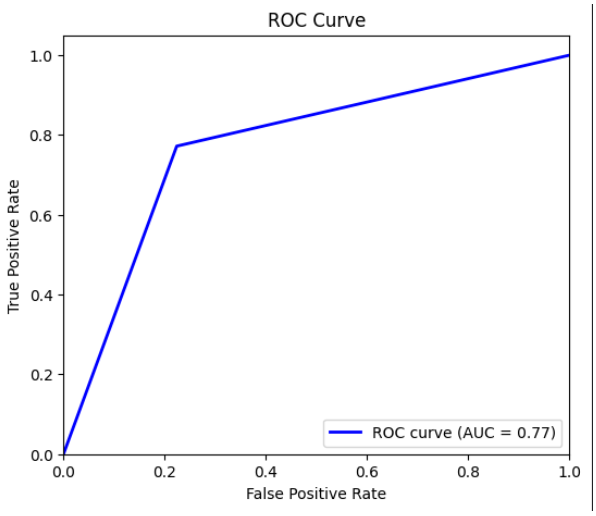
demonstrating the effectiveness of the applied text preprocessing techniques. Specifically, achieved an accuracy of 76,48% on the training set and 77,13% on the validation set. These results highlight the importance of thoughtful text preprocessing in sentiment classification tasks, as it helps reduce noise, enhances signal quality, and ultimately supports better generalization by the model.

	precision	recall	f1-score	support
0	0.75	0.78	0.77	20384
1	0.79	0.76	0.78	22012
accuracy			0.77	42396
macro avg	0.77	0.77	0.77	42396
weighted avg	0.77	0.77	0.77	42396

(a) Classification Report And Confusion Matrix



(b) Learning curve



(c) ROC curve

3.1.3. Experiment 3 - Number of Hidden Layers. The third experiment focused on examining the effect of network depth by varying the number of hidden layers within the neural architecture. Specifically, two models were evaluated: one consisting of three

hidden layers and another consisting of five. The primary objective of this experiment was to determine how increasing the depth of the model influences its ability to capture complex patterns in the data and whether deeper architectures give significant improvements in classification performance.

While deeper networks generally provide more capacity to learn abstract representations, this does not always translate into better performance—especially when the complexity of the task or the dataset does not require it.

In this case, the model with three hidden layers achieved better overall accuracy compared to the configuration with five layers. The deeper architecture did not result in performance improvement and, in fact, slightly underperformed during validation. This suggests that, for the current dataset and task complexity, the additional layers introduced unnecessary complexity without offering tangible benefits.

The three-layer model achieved noticeably improved performance compared to the five-layer version, demonstrating that a simpler architecture can sometimes be more effective. Specifically, they achieved an accuracy of 76.45% and 72,27% on the training set and 77.78% and 75,57% on the validation set. These results confirm that for the current dataset, a moderate level of depth is sufficient for capturing the underlying patterns, without adding unnecessary complexity or leading to reduced performance due to the extra depth.

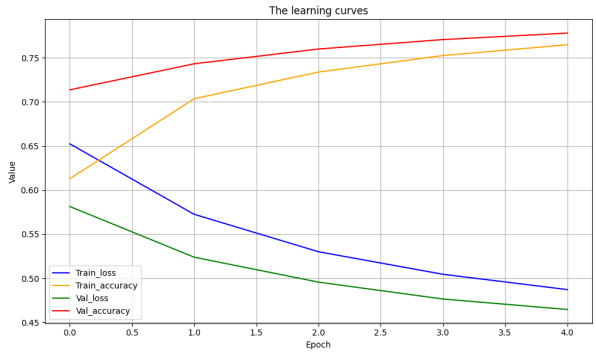
These findings highlight the importance of matching model depth to the nature of the data. In scenarios where the patterns to be learned are not overly complex, shallower models can achieve equal or even superior performance while maintaining lower computational cost and faster training times.

	precision	recall	f1-score	support
0	0.82	0.76	0.79	23066
1	0.73	0.80	0.77	19330
accuracy			0.78	42396
macro avg	0.78	0.78	0.78	42396
weighted avg	0.78	0.78	0.78	42396

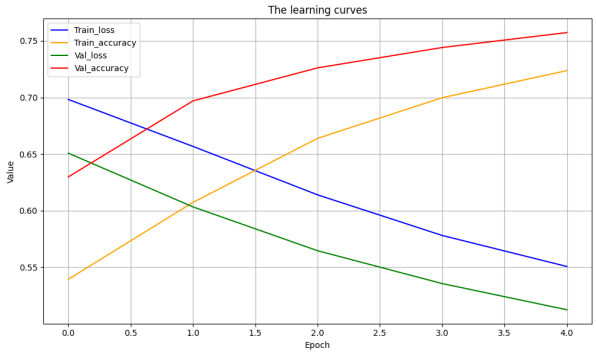
(a) Classification Report And Confusion Matrix on model 3.1

	precision	recall	f1-score	support
0	0.76	0.75	0.76	21324
1	0.75	0.76	0.75	21072
accuracy			0.76	42396
macro avg	0.76	0.76	0.76	42396
weighted avg	0.76	0.76	0.76	42396

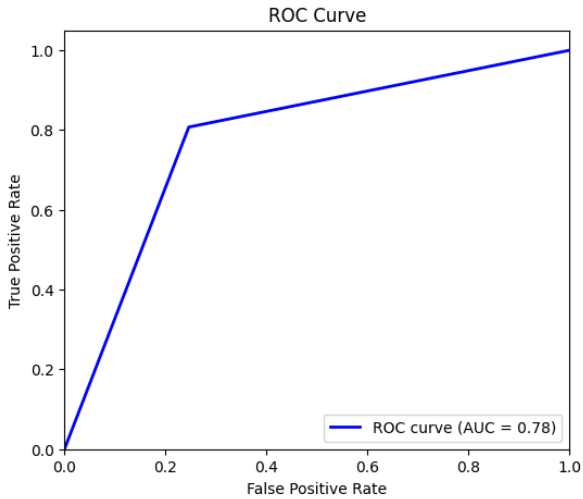
(b) Classification Report And Confusion Matrix on model 3.2



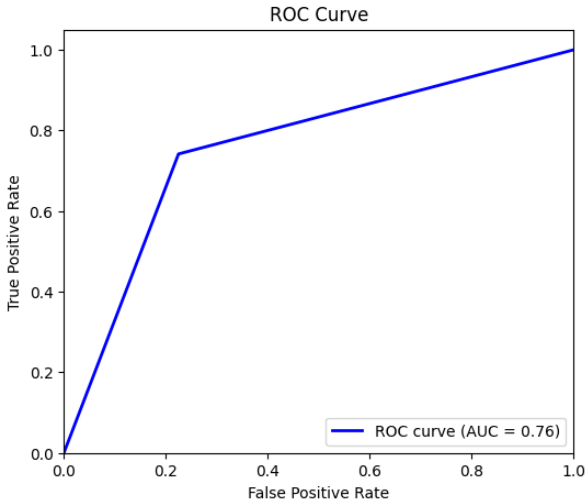
(a) Learning Curves on model 3.1



(b) Learning Curves on model 3.2



(a) Roc curve on model 3.1



(b) Roc curve on model 3.2

3.1.4. Experiment 4 – Number of Neurons per Hidden Layer. The fourth experiment was dedicated to exploring the impact of varying the number of neurons in each hidden layer. Two model architectures were constructed and evaluated: one utilizing what was determined to be an optimal configuration of neurons per layer, and another employing with a non optimized one.

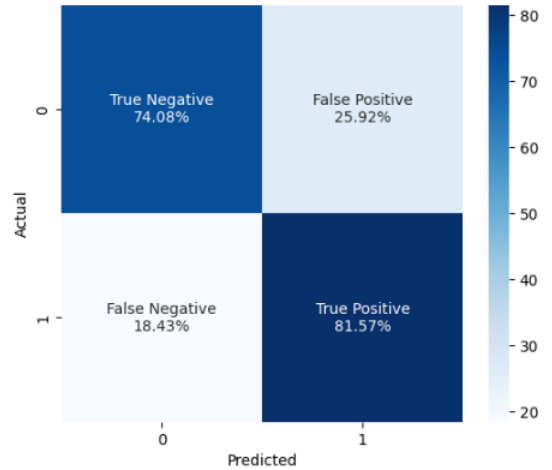
In both models, the number of neurons was systematically reduced across successive hidden layers following a decrease by a factor of two at each layer (i.e., powers of two). This strategy reflects a commonly adopted design pattern in deep learning, where the first layers capture more general, high-dimensional features, while the deeper layers progressively transform this information into more abstract, task-specific representations.

The aim of this experiment was to: first of all, determine how the number and distribution of neurons affect the model's ability to learn meaningful representations from the data, and second, evaluate whether a more compact architecture could maintain or even improve performance by reducing overfitting and computational overhead.

Specifically, the model configured with the more carefully selected neuron distribution(e.g., 512) achieved an accuracy of 76,29% on the training dataset and 77,32% on the validation dataset. In contrast, the alternative model with a less sophisticated configuration(e.g., 64) reached an accuracy of only 65,36% on the training set and 71,90% on the validation set.

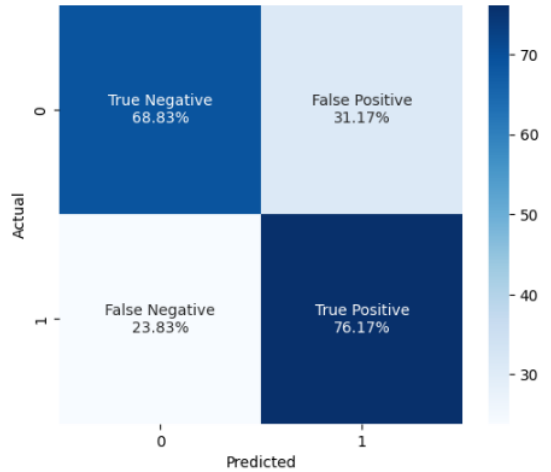
These results suggest that a well-structured allocation of neurons not only enhances the model's capacity to learn from data but also improves its generalization performance. The more balanced configuration appears to provide a better trade off between representational power and overfitting control, where the suboptimal configuration either failed to capture enough complexity or led to early stagnation during training.

	precision	recall	f1-score	support
0	0.84	0.74	0.79	24049
1	0.71	0.82	0.76	18347
accuracy			0.77	42396
macro avg	0.77	0.78	0.77	42396
weighted avg	0.78	0.77	0.77	42396

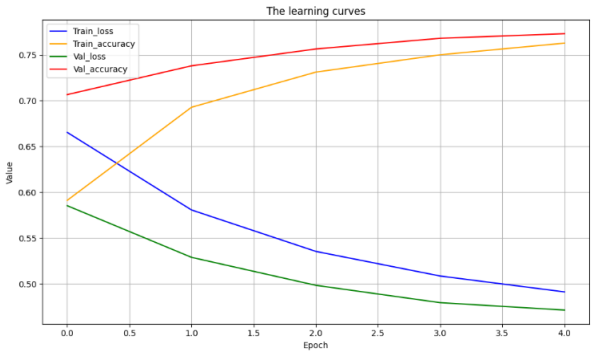


(a) Classification Report And Confusion Matrix on model 4.1

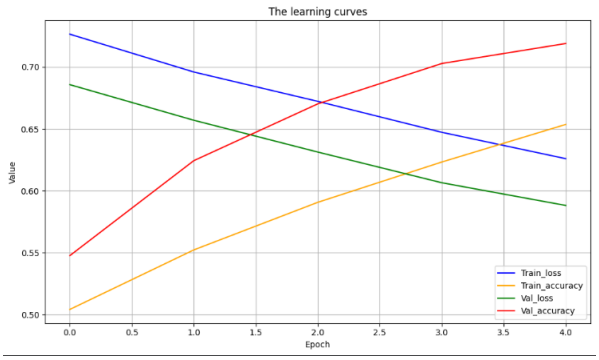
	precision	recall	f1-score	support
0	0.80	0.69	0.74	24652
1	0.64	0.76	0.69	17744
accuracy			0.72	42396
macro avg	0.72	0.72	0.72	42396
weighted avg	0.73	0.72	0.72	42396



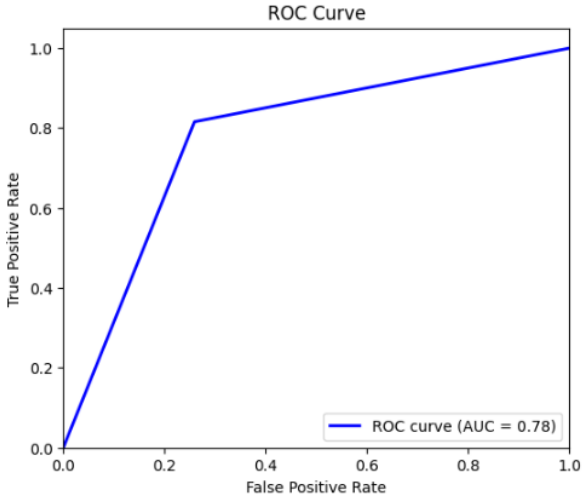
(b) Classification Report And Confusion Matrix on model 4.2



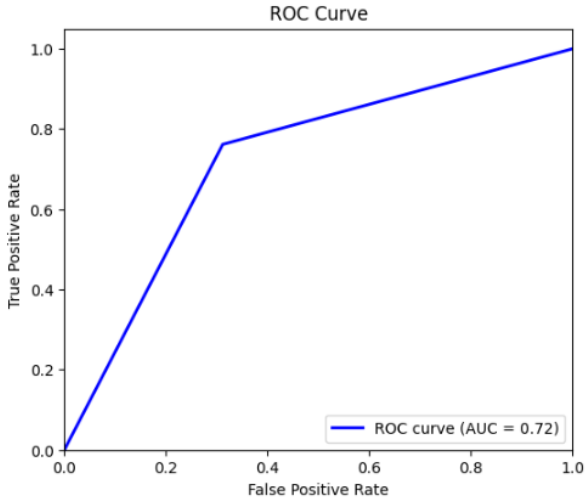
(a) Learning Curves on model 4.1



(b) Learning curves on model 4.2



(a) Roc curve on model 4.1

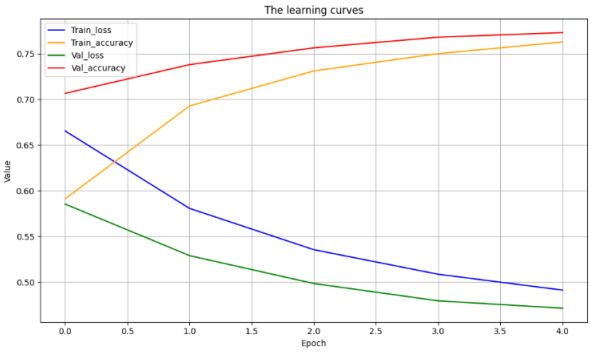


(b) Roc curve on model 4.2

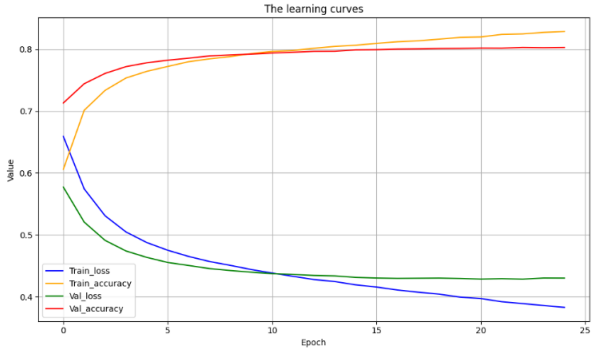
3.1.5. Experiment 5 – Number of Epochs. This experiment focused on how the number of training epochs affects the model's performance. Three different configurations were tested: one model trained for 25 epochs, which was found to be the most suitable during experimentation, another trained for only 5 epochs to simulate undertraining, and a third trained for 29 epochs to observe the effects of longer training.

Training for too few epochs may result in the model not learning enough from the data, while training for too many epochs may lead to overfitting, where the model memorizes the training data and performs poorly on new, unseen data. The aim was to find a balance between learning enough patterns from the training data and maintaining good generalization on the validation set.

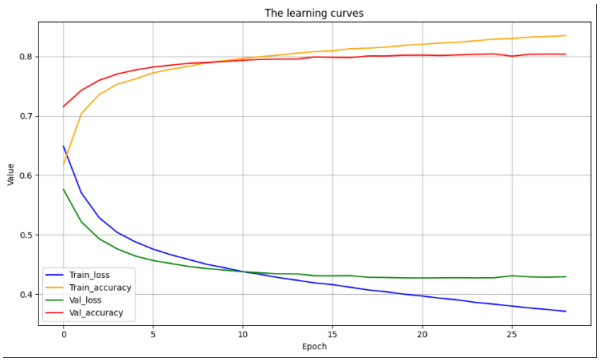
Specifically, the model trained for the optimal number of epochs(25) achieved an accuracy of 82,84% on the training dataset and 80,26% on the validation dataset. The undertrained model with only 5 epochs reached an accuracy of 76,29% on training and 77,32% on validation, showing clear signs of underfitting. On the other hand, the overtrained model with 29 epochs achieved 83,59% on training and 80,37% on validation, indicating that the model started to overfit the training data.



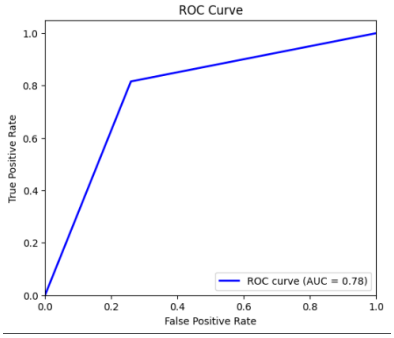
(a) Learning curves on model 4.1



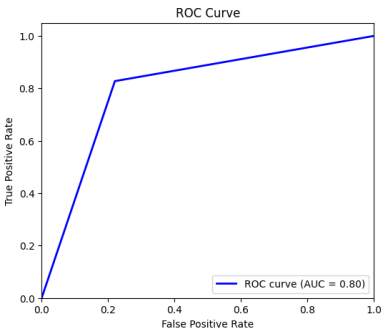
(b) Learning curves on model 5.1



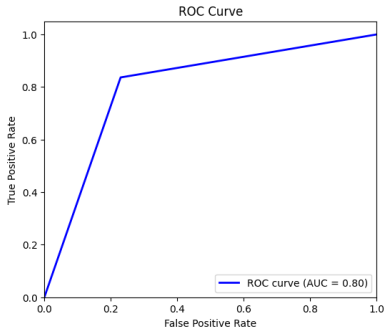
(c) Learning curves on model 5.2



(a) ROC curve on model 4.1

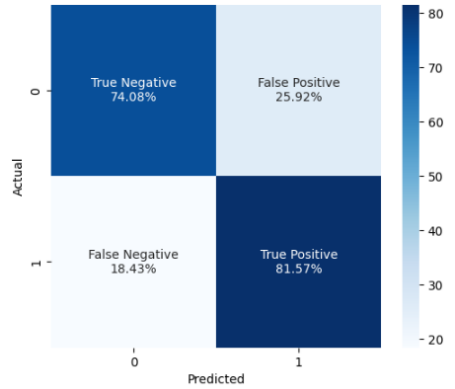


(b) ROC curve on model 5.1



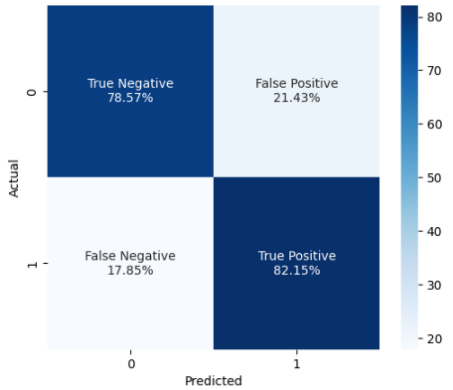
(c) ROC curve on model 5.2

	precision	recall	f1-score	support
0	0.84	0.74	0.79	24049
1	0.71	0.82	0.76	18347
accuracy			0.77	42396
macro avg	0.77	0.78	0.77	42396
weighted avg	0.78	0.77	0.77	42396



(a) Classification Report And Confusion Matrix on model 4.1

	precision	recall	f1-score	support
0	0.83	0.79	0.81	22448
1	0.77	0.82	0.80	19948
accuracy			0.80	42396
macro avg	0.80	0.80	0.80	42396
weighted avg	0.80	0.80	0.80	42396



(b) Classification Report And Confusion Matrix on model 5.1

	precision	recall	f1-score	support
0	0.83	0.79	0.81	22139
1	0.78	0.82	0.80	20257
accuracy			0.80	42396
macro avg	0.80	0.80	0.80	42396
weighted avg	0.80	0.80	0.80	42396

(c) Classification Report And Confusion Matrix on model 5.2

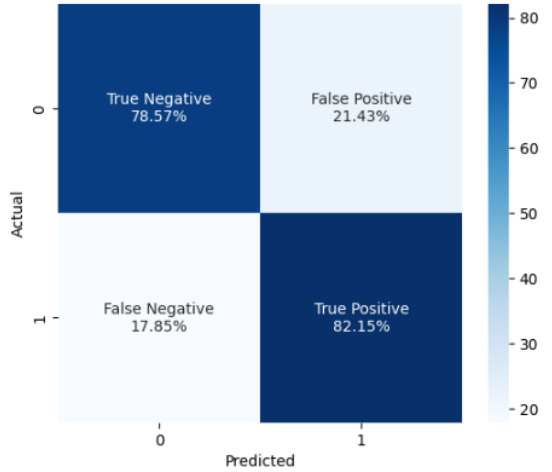
3.1.6. Experiment 6 – Learning Rate and Weight Decay. The sixth experiment aimed to investigate the impact of learning rate and weight decay on the model's training dynamics and overall performance. These two hyperparameters play a crucial role in optimizing neural networks, as they influence how quickly and how effectively the model converges to a minimum of the loss function.

In this experiment, two models were trained: the first used selected well-balanced values for learning rate and weight decay, while the second model was configured with suboptimal values for both. The goal was to observe how incorrect hyperparameters might hinder learning or lead to unstable training behavior.

A good learning rate helps the model learn efficiently without missing the optimal solution, where weight decay helps prevent overfitting by penalizing large weights during training. When these hyperparameters are poorly tuned, the model may either fail to learn or exhibit unstable loss curves.

Specifically, the model trained with the optimal learning rate and weight decay achieved a training accuracy of 82,84% and a validation accuracy of 80,26%. In contrast, the model using improper hyperparameters achieved 91,20% on the training set and only 72,85% on the validation set. The latter model also showed signs of overfitting during training, confirming the importance of careful hyperparameter tuning.

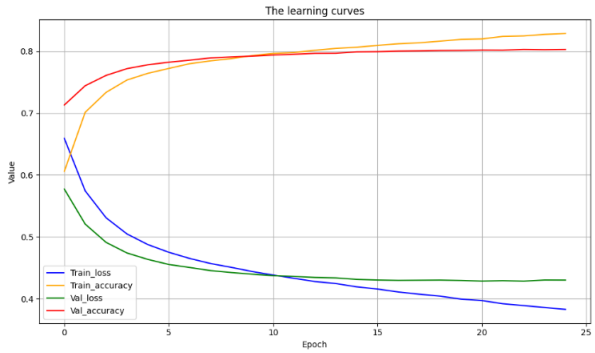
	precision	recall	f1-score	support
0	0.83	0.79	0.81	22448
1	0.77	0.82	0.80	19948
accuracy			0.80	42396
macro avg	0.80	0.80	0.80	42396
weighted avg	0.80	0.80	0.80	42396



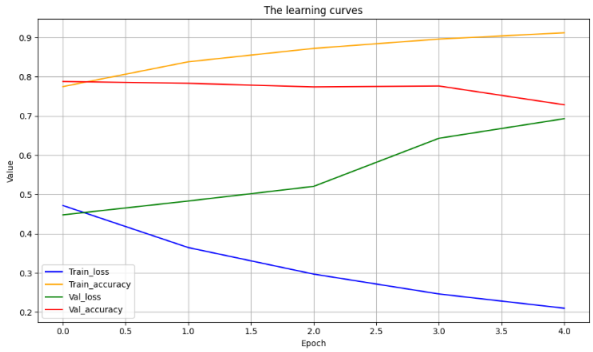
(a) Classification Report And Confusion Matrix on model 5.1

	precision	recall	f1-score	support
0	0.78	0.71	0.74	23326
1	0.68	0.75	0.71	19070
accuracy			0.73	42396
macro avg	0.73	0.73	0.73	42396
weighted avg	0.73	0.73	0.73	42396

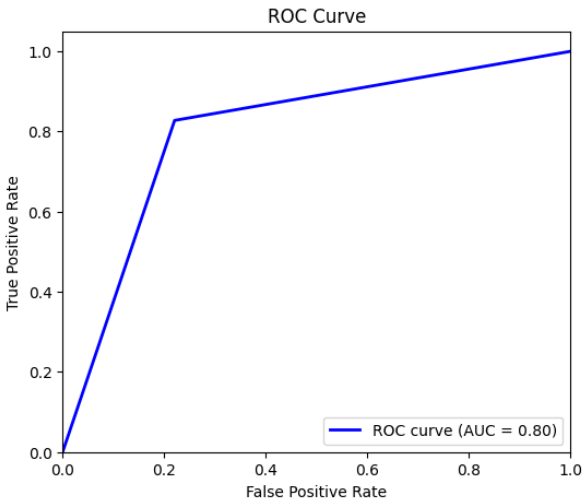
(b) Classification Report And Confusion Matrix on model 6



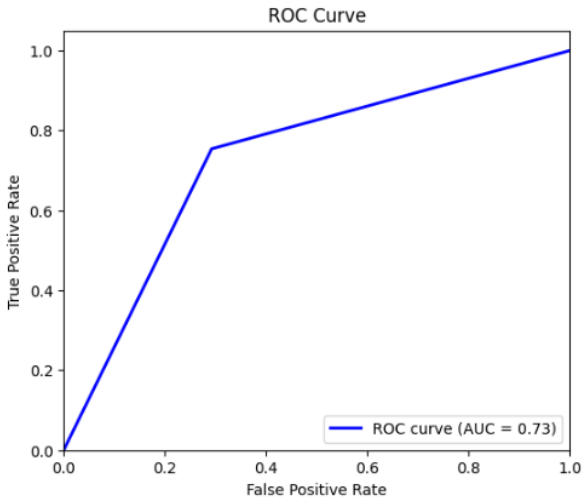
(a) Learning Curves on model 5.1



(b) Learning curves on model 6



(a) Roc curve on model 5.1



(b) Roc curve on model 6

3.1.7. Table of trials. The following table summarizes the different experiments conducted, including variations in text preprocessing, hyperparameters, and resulting performance metrics.

Trial	TP	HL	NPL	EP	LR/WD	TA	VA
E1	No	1	200	5	1e-5/1e-7	74,95%	74,46%
E2	Yes	1	200	5	1e-5/1e-7	76,48%	77,13%
E3.1	Yes	3	512	5	1e-5/1e-7	76,45%	77,78%
E3.2	Yes	5	512	5	1e-5/1e-7	72,27%	75,57%
E4.1	Yes	3	512	5	1e-5/1e-7	76,29%	77,32%
E4.2	Yes	3	64	5	1e-5/1e-7	65,36%	71,90%
E5.1	Yes	3	512	25	1e-5/1e-7	82,84%	80,26%
E5.2	Yes	3	512	29	1e-5/1e-7	83,59%	80,37%
E6	Yes	3	512	5	1e-2/1e-4	91,20%	72,85%

Table 1: Summary of the trials conducted and their results

*The "E" in the trials stands for Experiment, TP stand for Text Preprocessing, HL stands for Hidden Layers, NPL stands for Neurons per layer divided by a power of 2, EP stands for Epochs, LR/WD stands for Learning Rate/Weight Decay, TA stands for Training Accuracy and VA stands for Validation Accuracy.

3.2. Hyper-parameter tuning

Hyperparameter tuning is a crucial step in optimizing the model's performance, as it directly impacts both its ability to learn from data and its generalization to unseen samples. This section outlines the most important hyperparameters and presents the final selection of the optimal values.

3.2.1. Key Hyper-parameters. During hyperparameter tuning, the most important parameters that were considered are the following:

- **Hidden layers**: The number of hidden layers defines how deep the neural network is. More layers allow the model to learn more complex patterns, but also make training slower and potentially more difficult.
- **Number of neurons**: This is the number of units in each hidden layer. More neurons give the model more capacity to learn, but also increase the risk of overfitting and require more computational resources.
- **Batch size**: This determines how many samples are used to calculate and update the model's weights in each training step. Smaller batch sizes can help generalization, while larger ones can make training more stable and faster on some hardware.
- **Loss function**: The loss function measures how far the model's predictions are from the correct values. It is used to guide the learning process.
- **Optimizer**: The optimizer decides how the model's weights are updated during training. Popular choices like Adam or SGD have different strengths depending on the task and data.

- **Learning rate**: This controls how big the updates to the model's weights are. A learning rate that is too high might make the model unstable, while one that is too low can make training very slow.
- **Weight decay**: A technique that helps reduce overfitting by discouraging the model from assigning too much importance to any one feature (i.e., by keeping weights small).
- **Epochs**: The number of times the model goes through the entire training dataset. More epochs allow the model to learn better, but too many may lead to overfitting.
- **Activation function**: Determines the non-linear transformation applied to the output of each neuron. It directly affects how the model learns and represents complex patterns. Common choices include ReLU, sigmoid, and tanh.
- **Dropout rate**: A regularization method where a fraction of neurons is randomly deactivated during training. This helps prevent overfitting and improves the model's ability to generalize to unseen data.

3.2.2. Hyper-parameters Tuning Results. The results of the hyper-parameter tuning process can be grouped into two main categories: parameters with an easier optimal choice based on prior knowledge, and parameters that required more extensive experimentation to fine-tune.

In the first category, the dropout rate and activation function were selected based on widely accepted practices in deep learning. The dropout rate was set to 0.3, as values between 0.2 and 0.5 are generally considered effective in reducing overfitting by preventing co-adaptation of neurons. The ReLU activation function was chosen due to its simplicity and proven effectiveness in training deep neural networks. It helps the model learn faster while avoiding problems like vanishing gradients, which can happen with functions like sigmoid or tanh.

In the second category, parameters such as the number of hidden layers, number of neurons, batch size, loss function, optimizer, learning rate, weight decay, and number of epochs were fine-tuned through a combination of manual experimentation and automated search using Optuna, as discussed in the following section.

After thorough evaluation, the final configuration included a network with three hidden layers, starting with 512 neurons in the first layer, and reducing by half in each layer (i.e., 512-256-128). This structure offered a good balance between model capacity and generalization. The number of training epochs was set to 25, as it provided optimal performance without signs of overfitting. A batch size of 256 was selected since it consistently led to better training stability and faster convergence compared to smaller or larger sizes.

For the optimization setup, the BCELoss was used as the loss function, appropriate for binary classification tasks. The optimizer of choice was AdamW, which combines the benefits of Adam with improved weight decay regularization. A learning rate of 1e-5 and weight decay of 1e-7 were found to provide a stable training process, avoiding both overfitting and underfitting, while achieving solid generalization performance.

This combination of hyper-parameters was determined to be the most effective for the specific dataset and task, with consistent improvements in validation accuracy and overall model robustness.

3.2.3. Prevention of overfitting and underfitting. Through the above-mentioned hyperparameters, the model was able to achieve a good balance between overfitting and underfitting, resulting in stable performance across different datasets. By carefully tuning the key parameters, the model retained enough complexity to capture meaningful patterns without being overly sensitive to noise. This approach allowed for a more robust model capable of generalizing well. Both overfitting and underfitting will be discussed in more detail and analyzed in the upcoming evaluation section.

3.3. Optimization techniques

The optimization techniques used in this project aimed to identify the most effective configuration of the model's hyperparameters, ensuring both strong predictive performance and the ability to generalize well to unseen data. To achieve this, two strategies were applied: manual tuning and automated optimization using the Optuna framework.

The process began with a manual tuning phase, which served both as an initial exploration and as a way to build a deeper understanding of the model's behavior. At this stage, experimentation focused on basic and widely studied hyperparameters. For example, the dropout rate and the activation function as mentioned in the previous section.

Once a baseline understanding was established, the Optuna framework was employed to perform a more systematic search across a broader hyperparameter space. Optuna uses a form of Bayesian optimization, which allows for efficient exploration by focusing on promising parameters of the search space. The objective function was defined to maximize validation accuracy, which was selected as the main evaluation metric due to its direct alignment with classification performance.

During experimentation, it became evident that relying solely on validation loss could be misleading. While a lower loss often indicates more confident predictions, it does not always translate to better classification accuracy, especially in cases where the model becomes overconfident. For this reason, accuracy was prioritized as a more meaningful indicator of generalization.

After Optuna provided the best-performing hyperparameter combinations, a final tuning phase followed. This involved small manual adjustments based on insights from previous results, such as fine-tuning the number of epochs and observing the model's learning curves. Special attention was given to identifying the point where training accuracy continued to improve but validation accuracy began to stagnate or decrease potentially overfitting. In these cases, techniques like early stopping or reduced training epochs were used to ensure better generalization.

In conclusion, the combination of manual exploration and automated search allowed for a thorough and balanced optimization process. Each phase contributed complementary insights, ultimately leading to a well-tuned model with stable and reliable performance on both training and validation data.

3.4. Evaluation

Evaluating the model's performance is a crucial step in understanding its effectiveness in classification tasks. To assess the quality of predictions, multiple evaluation metrics and visualizations are used. These metrics help determine whether the model

generalizes well to unseen data, suffers from overfitting, or struggles with class imbalances.
The evaluation process includes:

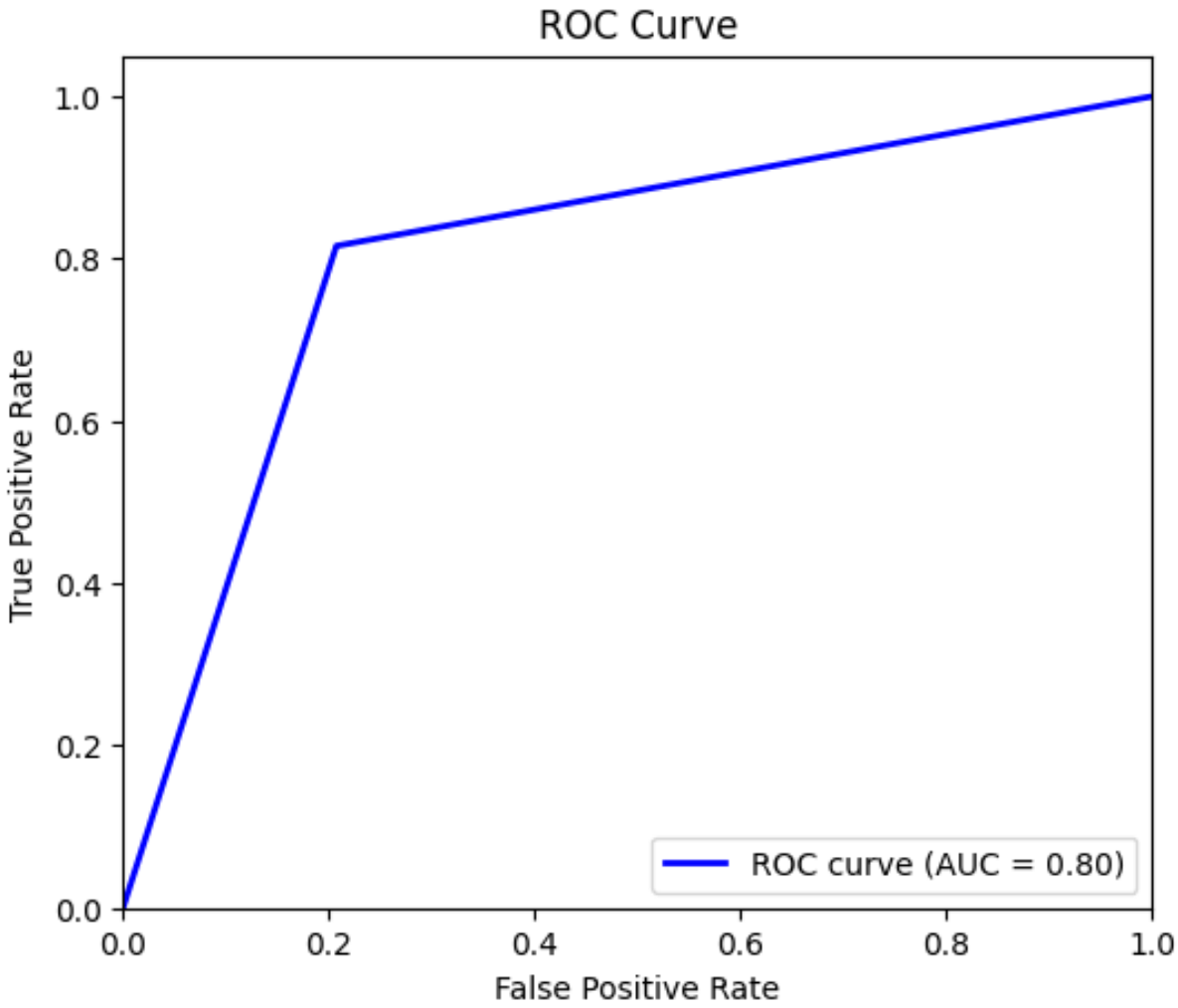
- **ROC Curve (Receiver Operating Characteristic Curve)**: Measures the model's ability to distinguish between classes across different threshold values.
- **Learning Curve**: Tracks the model's performance over time by plotting the training and validation loss and accuracy. It provides insights into the model's progress during training, helping to detect overfitting or underfitting.
- **Classification Report**: Provides a detailed breakdown of key classification metrics such as precision, recall, and F1-score.
- **Confusion Matrix**: Gives insights into the model's misclassifications by showing the distribution of true positives, false positives, true negatives, and false negatives.

Each of these evaluation tools plays a key role in analyzing the strengths and weaknesses of the model. In the following subsections, these metrics and visualizations will be discussed in detail, along with their respective results and interpretations.

3.4.1. ROC curve. The Receiver Operating Characteristic (ROC) curve is a fundamental evaluation tool for assessing the performance of a classification model. It illustrates the trade-off between True Positive Rate (TPR) and False Positive Rate (FPR) across different decision thresholds. A well-performing model will have a curve that approaches the top-left corner, indicating a high true positive rate with a low false positive rate.

To quantify the model's ability to distinguish between classes, the Area Under the Curve (AUC-ROC) is used. A value of 1.0 represents a perfect classifier, whereas 0.5 indicates a model that is no better than random guessing.

The ROC curve for the model is presented. The AUC score achieved was 0.80, indicating moderate to strong discriminative power. The curve demonstrates a steep rise initially, followed by a more gradual increase, suggesting that the model effectively distinguishes between classes at lower false positive rates but struggles slightly as the threshold increases.

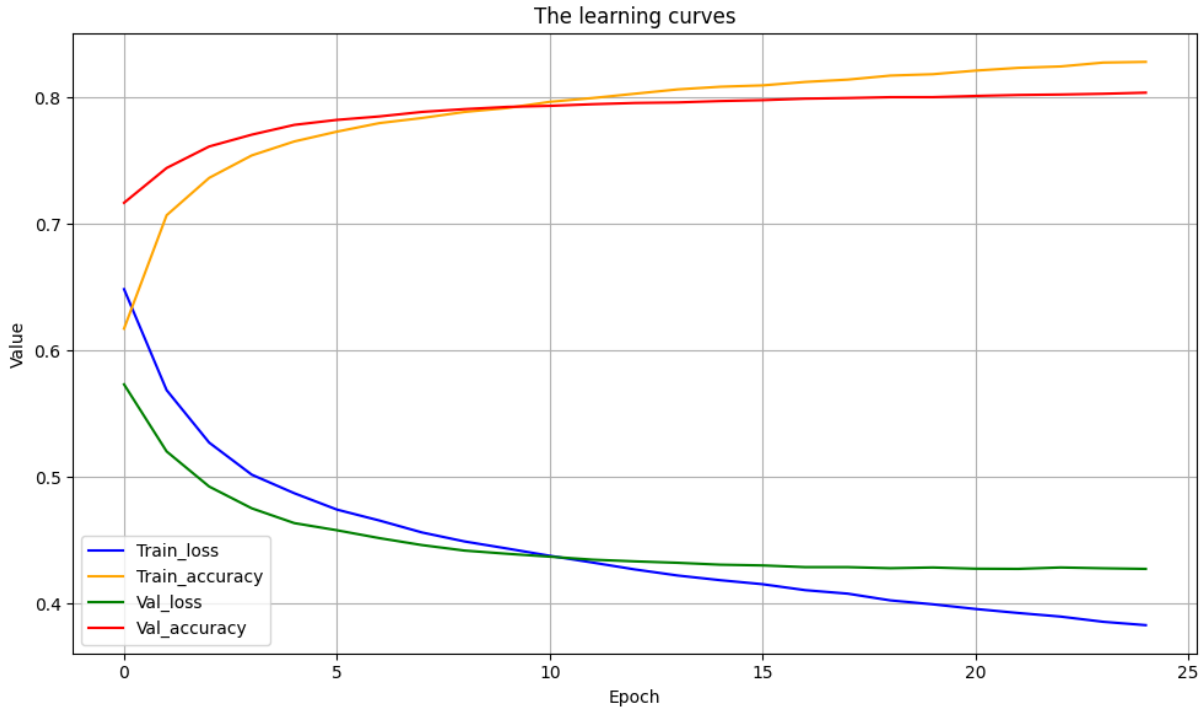


3.4.2. Learning Curve. The learning curve of the model is presented to illustrate how the training and validation loss evolved over time across epochs. This visualization provides valuable insight into how well the model learned from the training data and how effectively it generalized to unseen validation data.

Throughout training, the training loss showed a steady and consistent decrease, while the validation loss followed a similar downward trend with no significant divergence between the two. This pattern suggests that the model was able to extract meaningful patterns from the data without memorizing it, which would have resulted in overfitting.

Importantly, no signs of overfitting or underfitting were observed. The validation loss continued to decrease in parallel with the training loss, indicating that the model maintained its ability to generalize well. In cases of overfitting, we would expect the validation loss to increase or stagnate while the training loss keeps decreasing. On the other hand, if the model were underfitting, both losses would remain high or fluctuate unpredictably. Neither of these issues occurred, confirming that the training process was well-balanced.

Overall, the learning curve demonstrates that the model achieved stable and generalizable learning, without underfitting or overfitting. This reflects a well-tuned architecture and an effective optimization process.



3.4.3. Classification report. The classification report of the model presents key performance metrics, including precision, recall, and F1-score for both classes. The model achieves an accuracy of 80%, indicating a well-balanced predictive performance across the dataset.

For class 0, the model exhibits a precision of 0.82, meaning that when it predicts class 0, it is correct 82% of the time. Its recall is 0.79, signifying that it successfully identifies 79% of all actual class 0 instances. The F1-score is 0.81, representing the harmonic mean between precision and recall.

For class 1, the precision is 0.78, meaning that 78% of the predicted class 1 instances are correct. The recall for this class is 0.82, meaning the model correctly captures 82% of the actual class 1 instances. The corresponding F1-score is 0.80, indicating a good balance between precision and recall.

The macro average F1-score is 0.80, treating both classes equally, while the weighted average F1-score is also 0.80, accounting for class distribution. These results suggest that the model does not favor one class over the other and maintains consistent performance across both categories.

Overall, the classification report confirms that the model is well-calibrated, with no significant signs of overfitting or underfitting. The precision-recall balance across both classes and the stability of F1-scores indicate that the model generalizes effectively on unseen data.

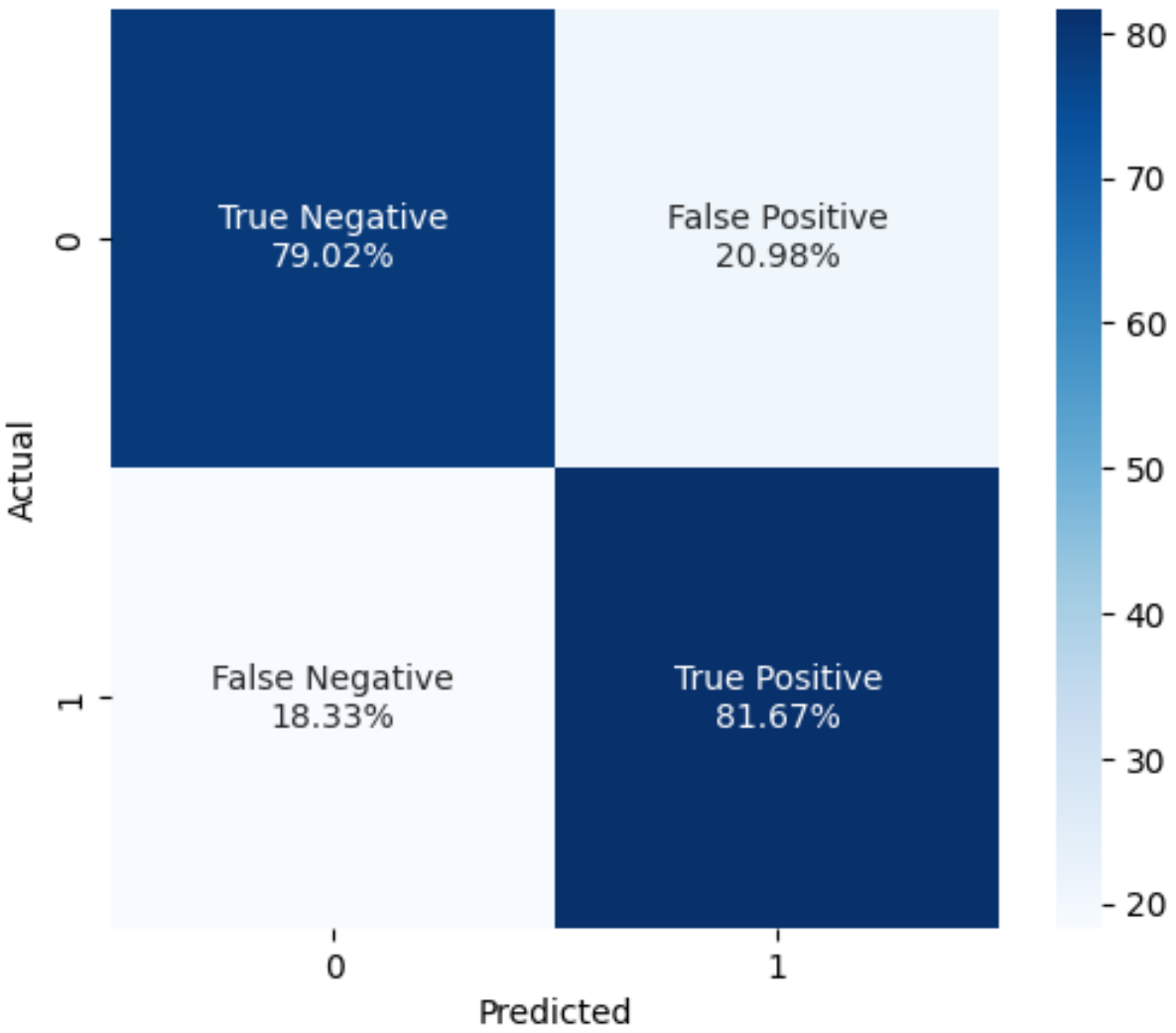
	precision	recall	f1-score	support
0	0.82	0.79	0.81	22123
1	0.78	0.82	0.80	20273
accuracy			0.80	42396
macro avg	0.80	0.80	0.80	42396
weighted avg	0.80	0.80	0.80	42396

3.4.4. Confusion matrix. The confusion matrix provides an in-depth look at the model's classification performance by breaking down its predictions into four categories:

- **True Negatives (TN)**: 79.02% of actual class 0 instances were correctly classified as 0.
- **False Positives (FP)**: 20.98% of actual class 0 instances were misclassified as 1.
- **False Negatives (FN)**: 18.33% of actual class 1 instances were misclassified as 0.
- **True Positives (TP)**: 81.67% of actual class 1 instances were correctly classified as 1.

These results suggest that the model has a balanced performance across both classes, with slightly better recall for class 1 (81.67%) compared to class 0 (79.02%). The false positive rate (20.98%) and false negative rate (18.33%) indicate that there are some misclassifications, but they remain within an acceptable range.

The overall structure of the confusion matrix, in conjunction with the classification report, reinforces that the model is well-trained, does not suffer from overfitting or underfitting, and is capable of effectively distinguishing between the two classes.



4. Results and Overall Analysis

4.1. Results Analysis

Following the systematic development of the model including thorough text preprocessing, feature engineering, and hyperparameter tuning the final results provide a clear picture of its performance. The ROC Curve, with an AUC score of 0.80, indicates a strong ability to differentiate between the two classes. Furthermore, the learning curve analysis confirms that the model is well trained, showing no signs of underfitting or overfitting, as both the training and validation scores converge smoothly. The classification report supports these findings, with balanced precision, recall, and F1 score, demonstrating a well-calibrated model with consistent performance across metrics. Additionally, the confusion matrix highlights that the true positive and true negative rates are well-balanced, with minimal misclassification.
Although the results are strong, further experiments could be conducted to explore potential improvements:

- **Word Embeddings**: Experimenting with more sophisticated word embedding techniques such as contextualized embeddings (e.g., BERT, RoBERTa) could lead to improved performance by capturing richer semantic relationships in the text.

- **Alternative Models**: Testing alternative architectures, such as LSTM based models, convolutional networks, or transformer-based models, may uncover architectures that better capture the sequential or contextual nature of the data.
- **Data Number**: Increasing the dataset size by collecting more labeled examples or performing data augmentation could enhance the models ability to generalize and reduce potential biases.

4.1.1. Best trial. The best trial achieved in this project represents a balance between strong performance metrics and the absence of overfitting. Several steps were taken to refine the final model, all of which have been thoroughly explained in the previous sections, accompanied by detailed justifications and evaluation plots.

After implementing these steps, the best model obtained is as follows:

- **Text Pre-processing**: The text preprocessing , as described in Section 2.1, ensured that the data was cleaned and transformed effectively before feature extraction.
- **Vectorization**: For text vectorization, pre-trained Word2Vec embeddings from the GloVe Twitter 200-dimensional dataset were utilized. The embedding matrix was constructed to include representations for out-of-vocabulary tokens as well as padding. Additionally, a normalization step was applied using a scaler to ensure consistent value ranges across embeddings.
- **Deep Learning Model**: The final model architecture consisted of three hidden layers with 512, 256, and 128 neurons respectively. Each layer was followed by a dropout layer with a rate of 0.3 and batch normalization to improve training stability and prevent overfitting. The model was trained using a batch size of 256, the Binary Cross Entropy(BCELoss) function, and the AdamW optimizer with a learning rate of 1e-5 and weight decay of 1e-7. Training was conducted over 25 epochs.
- **Overfitting and Underfitting Prevention**: Strategies to prevent overfitting and underfitting were discussed in Section 3.2.3, while the learning curve analysis in Section 3.4.2 further validated the model's ability to generalize well.
- **Model Evaluation**: The model's performance was assessed in Section 3.4, achieving strong results across multiple evaluation metrics, as presented in Section 3.4.3.

In conclusion, the model attained an accuracy of 82.87% on the training dataset, 80.29% on the validation dataset and 80.310% on the testing dataset.

4.2. Comparison with the first project

Comparing the results of this project with those from the previous one, we can see that there wasn't a significant improvement in performance. While there was a small increase in the test accuracy, the rest of the evaluation metrics stayed mostly the same.

This may be the outcome due to a few factors. First, although a more complex model was used this time, the dataset itself might not have been large or diverse enough to fully take advantage of that complexity. Also, while Word2Vec embeddings

added more semantic depth, they may not have made a big difference in this particular classification task, possibly because the dataset was already fairly clean.

In any case, this project helped build a stronger understanding of hyperparameter tuning, model optimization, and evaluation strategies. Hopefully, in the next project, with a new model approach, i will be able to push the performance further and achieve more noticeable improvements.

5. Bibliography

References

[1] GeeksforGeeks. Feedforward neural network, 2023. URL.

[2] Daniel Jurafsky and James H. Martin. *Speech and Language Processing(3rd edition draft)*. Stanford University, 2023. URL.

[3] Manolis Koumparakhs. Backpropagation, 2025. URL.

[4] Manolis Koumparakhs. Perceptrons, 2025. URL.

[5] Manolis Koumparakhs. Training deep neural networks, 2025. URL.

[6] Manolis Koumparakhs. Word vectors, 2025. URL.

[7] University of Illinois CS447. Lecture 20 - deep neural networks training, 2018. URL.

[8] PyTorch. Adamw, 2024. URL.

[9] PyTorch. Bceloss, 2024. URL.

[10] PyTorch. Pytorch, 2024. URL.

[11] Kamlesh Rangi. Feed forward neural network language model, 2022. URL.

[12] Kamlesh Rangi. How min-max scaler works, 2023. URL.

[13] Myrto Tsokanaridou. Hw2 tutorial 1, 2025. URL.

[14] Myrto Tsokanaridou. Hw2 tutorial 2, 2025. URL.